Explicación y defensa de la solución — Mantia Servicios S.A.S.

Prueba técnica — Desarrollo de software y resolución de problemas (SweetMeSoft) Prototipo: *MantiaService* (API .NET 10 + app Angular 20). Fecha de referencia del caso: **13 de octubre de 2026**.

> Este documento acompaña al repositorio. También existe una versión PDF en `docs/Solucion_Mantia.pdf`. El historial de interacción con la IA está en `docs/Historial_IA.md`.

---

1. Resumen ejecutivo

Mantia recibe solicitudes por correo, mensajería y documentos. Laura, la coordinadora comercial, invierte mucho tiempo en entender cada solicitud, consultar tarifas y preparar cotizaciones; algunas solicitudes se olvidan y otras se cotizan con información incompleta.

Construí un **prototipo local funcional** que:

1. **Recibe** una comunicación (texto pegado o archivo `.txt` / `.pdf`) y la convierte a texto.
2. **Interpreta** la solicitud y extrae los datos relevantes (cliente, ubicación, fecha, alcance, cantidades) usando un motor de extracción con **dos caminos**: Google Gemini y, como respaldo, un **motor de reglas local**.
3. **Clasifica** la solicitud en *Lista para cotizar*, *Falta información* o *Requiere valoración técnica*, señalando los campos faltantes.
4. **Cotiza** automáticamente aplicando las tarifas vigentes según la fecha de la visita, IVA, desplazamiento y recargos, generando un **borrador** que Laura revisa.
5. **Da visibilidad**: panel con pendientes, listado/filtro de solicitudes y cotizaciones, e historial conservado.

El resultado se verificó con las comunicaciones y tarifas reales del caso: los importes calculados y el historial sembrado **coinciden** con `02_Historial_Cotizaciones.xlsx`.

2. Interpretación del problema

El problema no es "redactar cotizaciones", sino **reducir trabajo manual y errores** en el tramo que va desde una comunicación informal del cliente hasta una cotización correcta.

Leí el caso como tres tareas distintas y encadenadas:

- **Ingesta heterogénea.** Las solicitudes llegan en formatos y canales distintos (correo, mensajería, PDF) y no siempre están completas. Hay correcciones posteriores (Colegio Arboleda pasa de 3 a 4 aires) y documentos que **duplican** una solicitud ya enviada (el PDF `SML-018` es soporte del correo de Edificio Mirador, no una solicitud nueva).

- **Interpretación con reglas de negocio.** "3 aires split de 18.000 BTU" debe mapearse a un código de tarifa (`AA-01`) y a una cantidad; "120 m² de pared" debe mapearse a `PI-01` cobrando **m² de pared, no de piso**; un "ascensor" **no** está en el listado y requiere valoración técnica; una "reparación sin ficha" tampoco se cotiza con el diagnóstico.
- **Cálculo auditable.** Precio por versión de tarifa vigente a la fecha de la visita, IVA 19%, un **solo** desplazamiento por visita, recargo por fin de semana/fuera de horario, y validez de 15 días. Todo debe ser trazable y revisable por Laura.

La frase guía de la empresa —"Necesita poder **entender y revisar** la información que utiliza para atender a cada cliente"— define el criterio de diseño: **automatizar la preparación, no la decisión**. Por eso la salida es siempre un **borrador revisable** y nunca un envío automático.

3. Alcance elegido

Dentro del alcance (implementado):

- Ingesta por texto o archivo (`.txt` / `.pdf`) simulando los canales; **no** hay conexión a cuentas reales de correo/mensajería (así lo permite la prueba).
- Interpretación de la solicitud y extracción de campos con Gemini + respaldo por reglas.
- Clasificación de estado y detección de campos faltantes.
- Registro, listado, filtro y edición de solicitudes.
- Cálculo de cotización (líneas, versión de tarifa, subtotal, IVA, total, validez), generación de borradores, listado/detalle e impresión.
- Panel de indicadores y consulta de tarifas por versión.
- Datos de ejemplo sembrados (tarifas, 4 solicitudes, 6 cotizaciones históricas).

Fuera del alcance (por decisión de la prueba o de tiempo):

- Integración real con correo/mensajería y envío real de cotizaciones.
- Agenda de técnicos, facturación/cobro.
- Autenticación/roles (uso local por Laura).
- Pruebas automatizadas formales (la verificación fue por API con los datos del caso).

Justifiqué este alcance porque cubre el **núcleo del dolor** (interpretar → cotizar con información completa) y deja como pendiente explícito lo que la prueba misma excluye (integraciones, agenda, cobro).

4. Cómo funciona la solución

4.1 Flujo de una solicitud



4.2 Decisión clave: IA *con* respaldo determinista

El caso exige que el sistema **no dependa de un servicio de pago** y que sea **ejecutable y comprensible**. Por eso el motor de extracción se diseñó como una interfaz (`IExtraccionService`) con dos implementaciones intercambiables:

- **`GeminiExtraccionService`** : llama a la API de Google Gemini con un prompt que fuerza una respuesta JSON estructurada.
- **`ReglasExtraccionService`** : heurísticas locales con expresiones regulares y normalización de texto (sin tildes, minúsculas).

`ExtraccionService` decide en caliente: si hay API key intenta Gemini; si falla (timeout, error HTTP, JSON inválido) **cae al motor de reglas** y registra la causa en el log. Esto garantiza que el prototipo funcione sin configuración y sin internet.

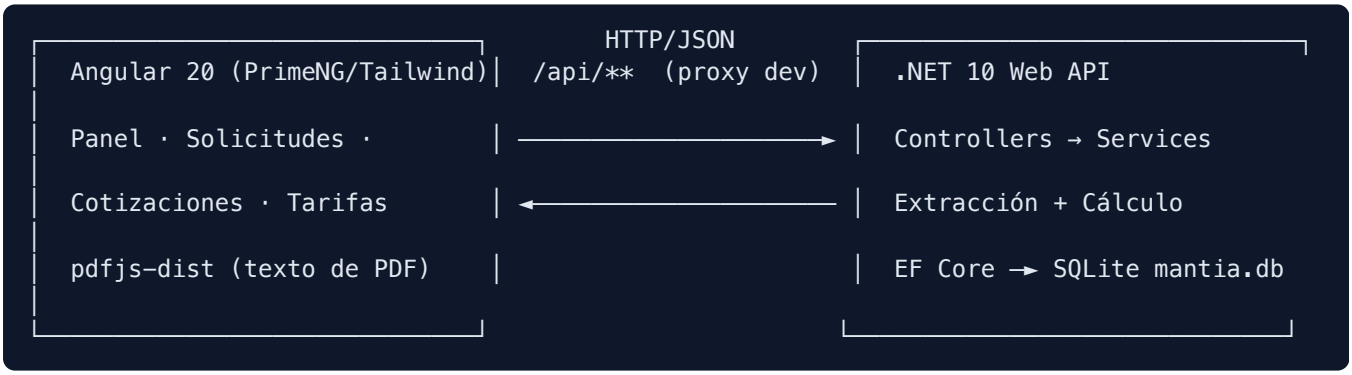
El cálculo de precios nunca usa IA. Es código determinista y auditable (`CotizacionCalculator` + `ReglasComerciales`), porque un valor mal calculado es exactamente el error que la empresa quiere evitar.

5. Arquitectura

5.1 Componentes

- **`MantiaServiceApi` (.NET 10, ASP.NET Core Web API)**
- `Controllers/`: `Solicitudes`, `Cotizaciones`, `Tarifas` (REST + Swagger).
- `Services/Extraccion/`: `IExtraccionService`, `ExtraccionService`, `GeminiExtraccionService`, `ReglasExtraccionService`, `ResultadoExtraccion`.
- `Services/Cotizacion/`: `CotizacionCalculator`, `CotizacionService`, `ReglasComerciales`, `ResultadoCalculo`.
- `Models/` + `DTOs/`: entidades EF Core y contratos de la API.
- `Data/`: `AppDbContext` (EF Core + SQLite) y `DataSeeder` (datos de ejemplo).
- `Configuration/EnvFile.cs`: carga simple de `.env` sin dependencias externas.
- `Migrations/`: esquema versionado de la base de datos.
- **`MantiaServiceApp` (Angular 20)**
- `core/services/`: `SolicitudesService`, `CotizacionesService`, `TarifasService`, `PdfService` (extracción de texto de PDF en el cliente).
- `features/`: `dashboard`, `solicitudes` (lista, nueva, detalle), `cotizaciones` (lista, detalle), `tarifas`.
- `shared/`: componentes de UI reutilizables, pipe `cop` (formato COP).
- `proxy.conf.json`: proxifica `/api` a `http://localhost:5256` en desarrollo.

5.2 Diagrama



5.3 Modelo de datos

- `Solicitud` (1) ——< `ItemSolicitud` — datos interpretados y estado.
- `Solicitud` (1) ——< `Cotizacion` (0..*) — una solicitud puede tener revisiones.
- `Cotizacion` (1) ——< `LineaCotizacion` — líneas con precio y valor.
- `Tarifa` — catálogo por `Version` + `Codigo`, con `Tipo` (`Servicio` / `Recargo` / `Desplazamiento`) y `PrecioUnitarioCOP`.

Se usan `decimal` (precisión 18,2) para montos y relaciones con borrado controlado (`Cascade` de ítems y líneas; `Restrict` de cotización→solicitud para conservar historial).

6. Reglas de negocio implementadas (y su origen)

Cada regla proviene de los documentos entregados:

Regla	Implementación
IVA 19% sobre servicios, recargos y desplazamiento	`ReglasComerciales.CalcularIva`
Desplazamiento Bogotá $30.000 / Chía $60.000 (2026-10), un solo cobro por visita	`CodigoDesplazamiento` + línea única
Recargo 25% sobre **servicios** si fin de semana o fuera de 8:00–18:00; **no** sobre desplazamiento	`AplicaRecargo`, `CalcularRecargoServicios`
"Urgente" no genera recargo	No hay regla que lo detecte (a propósito)
Versión de tarifa por fecha de visita: `2026-01` o `2026-10`	`ResolverVersionTarifa`
Visitas fuera de vigencia no se cotizan	Error en `CotizacionCalculator`
Validez 15 días calendario	`DiasValidez` → `ValidaHasta`
Ascensor / trabajo no listado / reparación sin ficha → valoración técnica	`ReglasExtraccionService` + guarda en el cálculo
Pintura cobra m² de pared, no de piso	Prompt y regla de extracción `PI-01`
Presupuesto del cliente no fija el precio	Observación + requiere valoración
No se envía automáticamente al cliente	Estado `Borrador`; "enviar" solo marca el estado

7. Motor de extracción

7.1 Prompt de Gemini (fragmento)

```
Extrae datos de una solicitud de mantenimiento para Mantia (Bogotá/Chía).
Devuelve SOLO JSON con esta forma:
{ "cliente", "contacto", "municipio", "direccion", "emailContacto", "referencia",
  "fechaVisita", "franjaHoraria", "estado", "camposFaltantes", "observaciones",
  "items": [{ "codigoTarifa", "concepto", "cantidad", "observacion" }] }
Reglas:
- AA-01 preventivo split <= 24000 BTU; AA-02 diagnóstico (no incluye reparación).
- PI-01 pintura interior m² de pared; PL-01 grifería sencilla; EL-01 punto eléctrico.
- Ascensor u otro trabajo no listado => RequiereValoracionTecnica y items vacíos.
- Reparación de aire sin ficha => RequiereValoracionTecnica.
- Si hay corrección de cantidad, usa la última. "urgente" no es recargo.
```

Se usa `responseMimeType: application/json` y `temperature: 0` para reducir variabilidad.

7.2 Motor de reglas local (respaldo)

Normaliza el texto (elimina tildes, pasa a minúsculas) y detecta:

- **Cliente/contacto** por patrones (`Soy …` , `De: …` , nombres conocidos del caso).
- **Ubicación** (Bogotá/Chía) y **dirección** (`Calle|Carrera …`).
- **Referencia** (`SML-018` , `ARR-042`) y **email**.
- **Agenda**: fecha y franja horaria; marca fin de semana y fuera de horario.
- **Ítems y estado** por palabras clave (`mantenimiento preventivo` , `pintar` , `grifer` , `punto eléctrico` , `ascensor` , `dejó de enfriar` , `arreglen` …), extrayendo cantidades y BTU.

8. Verificación de resultados (evidencia)

Todo se probó contra la API real con los datos entregados (motor de reglas, fecha de emisión 13/10/2026, tarifa `2026-10`).

8.1 Las 6 comunicaciones de ejemplo

Canal	Archivo	Resultado
Correo	`01_Correo…0815`	Colegio Arboleda, Bogotá, `AA-01` × 3, *Lista para cotizar*
Correo	`02_Correo…1010`	Edificio Mirador, Chía, ref `SML-018` , `PI-01` × 120 + `PL-01` × 2
Mensajería	`03_Mensajes…1430`	Oficina Horizonte, reparación sin ficha → *Requiere valoración técnica*
Mensajería	`04_Mensajes…0910`	Corrección: Colegio Arboleda, `AA-01` × 4 (usa la última cantidad)
PDF	`05_Solicitud_SML-018`	Misma solicitud del correo 02 → duplicado detectado por referencia
PDF	`06_Solicitud_Arrayanes`	Ascensor → *Requiere valoración técnica* (presupuesto del cliente no fija precio)

8.2 Cotizaciones calculadas

Solicitud	Detalle	Subtotal	IVA	Total
SOL-001	4 × 95.000 + desplazamiento Bogotá 30.000	410.000	77.900	**487.900**
SOL-002	120 × 14.000 + 2 × 65.000 + desplazamiento Chía 60.000	1.870.000	355.300	**2.225.300**
SOL-003	valoración técnica → **HTTP 422** con motivos	—	—	—
SOL-004	ascensor → **HTTP 422** con motivos	—	—	—

8.3 Casos de borde

- **Recargo fin de semana** (sábado): servicios 380.000 + recargo 95.000 + desplazamiento 30.000 → **600.950** total.
- **Recargo fuera de horario** (19:00–21:00): servicios 1.810.000 + recargo 452.500 + desplazamiento 60.000 → **2.763.775** total.
- **Visita fuera de vigencia de tarifas** (2027) → rechazada.
- **Historial:** las 6 cotizaciones sembradas coinciden exactamente con el XLSX (COT-001 295.000/56.050/351.050 … COT-006 297.500/56.525/354.025).

8.4 Builds

- Backend: `dotnet build` → compilación correcta (0 errores, 0 advertencias).
- Frontend: `npx ng build` → compilación correcta (aviso de presupuesto de bundle).

9. Herramientas y modelos elegidos

Área	Elección	Por qué
Backend	**.NET 10**	Tipado fuerte, EF Core, Swagger, despliegue simple en un solo comando
Persistencia	**SQLite**	Cero configuración; el prototipo arranca sin instalar motores
Frontend	**Angular 20**	Componentes, ruteo, DI y HttpClient; PrimeNG acelera una UI clara
PDF	**pdfjs-dist**	Extrae texto en el cliente, sin costo ni backend extra
IA	**Google Gemini** (`gemini-3.8-flash`)	JSON estructurado, tier gratuito, integración HTTP directa
Respaldo	**Reglas locales**	Garantiza funcionamiento sin API key / sin internet

10. Alternativas consideradas y descartadas

- **Solo reglas locales.** Descartada como única opción: robusta y gratis, pero frágil ante redacciones nuevas. Se conserva como **respaldo**; Gemini aporta generalización.
- **Solo LLM.** Descartada: introduce costo, latencia y no determinismo, y el caso pide no depender de un servicio de pago. Por eso el cálculo es 100% determinista.
- **Extracción de PDF en el servidor.** Descartada: añadía dependencias nativas; `pdfjs-dist` en el cliente es más simple y suficiente para documentos con texto (no escaneados).
- **Base de datos relacional en contenedor (Postgres/SQL Server).** Descartada por peso de instalación; SQLite basta para un prototipo local.
- **Microservicios / colas.** Descartadas: sobredimensionadas para el alcance y el tiempo.
- **Autenticación por roles.** Pospuesta: el prototipo es de uso local por una sola persona.

11. Limitaciones (explícitas)

- **Integraciones reales ausentes**: no se conectan cuentas de correo/mensajería ni se envían cotizaciones. La ingesta es manual (pegar/cargar).
- **Fecha de visita en el motor de reglas**: si el texto contiene varias fechas (encabezado del correo + fecha solicitada), la heurística puede tomar la primera. Gemini mitiga esto; aun así, Laura edita el campo. *Mejora pendiente: priorizar fechas junto a "visita", "solicitada" o días de la semana.*
- **Nombres de contacto**: la heurística puede acortar el nombre ("Ana" en vez de "Ana Beltrán"). El campo es editable.
- **PDF escaneados**: sin texto embebido, no se extraen (se avisa y se puede pegar el texto).
- **Duplicados**: se detecta la referencia repetida, pero la decisión de fusionar es manual.

- **Sin pruebas unitarias/e2e automatizadas** ni autenticación.
- **Cálculo**: redondeo a peso entero; no se manejan impuestos múltiples ni monedas.
- **Presupuesto del cliente** no se usa como precio (correcto según tarifas), pero tampoco se genera una alerta de comparación automática contra el cálculo.

12. Aportes de la IA: qué acepté, modifiqué o descarté

El detalle cronológico está en `docs/Historial_IA.md`. Resumen de criterio:

- **Acepté**: propuesta de arquitectura en dos capas con motor de extracción intercambiable (Gemini + reglas); estructura de entidades y DTOs; borradores de las `ReglasComerciales`.
- **Modifiqué**: los *prompts* (los forcé a JSON estricto con `temperature: 0` y reglas de negocio explícitas); las heurísticas de extracción (las alineé a los casos reales); la interpretación de fechas y del recargo (verificado contra el XLSX).
- **Descarté**: sugerencias de llamar a la IA para calcular precios; propuestas de microservicios/colas; extracción de PDF en servidor.
- **Cómo verifiqué**: calculé a mano los importes esperados de cada solicitud y de los casos de borde, y los comparé contra las respuestas reales de la API (sección 8). El historial de 6 cotizaciones funcionó como *conjunto de validación* del motor de cálculo.

13. Cómo reproducir la verificación

```
# 1) Backend (sin Gemini, para resultados deterministas)
cd MantiaServiceApi
GEMINI__APIKEY= dotnet run --urls http://localhost:5256

# 2) Analizar una comunicación
curl -s -X POST http://localhost:5256/api/solicitudes/analizar \
  -H "Content-Type: application/json" \
  -d '{"texto":"Mantenimiento preventivo de 3 aires split de 18000 BTU, Bogotá, Calle 120
# 15-20","origen":"Correo"}'

# 3) Generar cotización de la solicitud 1 (o 2)
curl -s -X POST http://localhost:5256/api/cotizaciones/solicitudes/1
```

14. Conclusión

La solución ataca el núcleo del problema con un diseño **simple, local y auditable**: interpretación asistida por IA con respaldo determinista, y decisión final en manos de Laura. El prototipo demuestra el flujo completo (solicitud → interpretación → clasificación → cotización → revisión) y sus resultados coinciden con los datos comerciales entregados. Las limitaciones y los pendientes están identificados de forma explícita y acotados al alcance permitido por la prueba.